

Sistema de E-commerce Distribuído com Mensageria

Caio Dalagnoli Dranka, Vinicius Muller

Departamento de Sistemas e Computação – Universidade Regional de Blumenau (FURB)

Resumo. *Este artigo descreve o desenvolvimento de um sistema distribuído de processamento de pedidos de e-commerce, construído com arquitetura orientada a mensagens. O sistema utiliza Amazon SQS (emulado via LocalStack) para comunicação assíncrona entre cinco serviços independentes, PostgreSQL para persistência, a API pública ViaCEP para validação de endereços, e Terraform para provisionamento da infraestrutura como código. Os resultados demonstram o funcionamento integrado dos componentes, incluindo tolerância a falhas via Dead Letter Queue.*

1. Introdução

O processamento de pedidos em sistemas de e-commerce envolve múltiplas etapas que, em uma arquitetura monolítica, tendem a se tornar acopladas e difíceis de escalar: validação de pagamento, controle de estoque, emissão de documentos fiscais, logística de entrega e comunicação com o cliente. Este trabalho propõe uma solução distribuída em que cada uma dessas etapas é tratada por um serviço independente, comunicando-se de forma assíncrona por meio de filas de mensagens.

O propósito do projeto é demonstrar, na prática, os conceitos fundamentais de sistemas distribuídos exigidos pela disciplina: o uso de mensageria como mecanismo de desacoplamento entre serviços, a persistência de dados em um banco de dados relacional preparado para concorrência, a integração com uma API externa para enriquecimento de dados, a existência de múltiplos serviços operando de forma independente, e o uso de infraestrutura de nuvem — neste caso emulada localmente através do LocalStack, com provisionamento via Terraform.

O desenvolvimento partiu de uma versão inicial do sistema baseada em RabbitMQ, posteriormente migrada para Amazon SQS de modo a atender ao requisito de uso de nuvem e permitir o gerenciamento da infraestrutura como código. As seções seguintes detalham a arquitetura da solução, as decisões técnicas tomadas durante a implementação e os resultados obtidos.

2. Desenvolvimento

2.1 Visão Geral da Arquitetura

O sistema é composto por uma API REST (serviço de pedidos), um conjunto de cinco filas SQS, cinco serviços consumidores independentes, um banco de dados PostgreSQL

e uma integração com a API pública ViaCEP. A Figura 1 (inserir diagrama de arquitetura gerado na etapa de definição do projeto) ilustra o fluxo: o cliente envia um pedido via HTTP; a API valida o CEP informado, persiste o pedido e publica a mensagem correspondente nas cinco filas; cada consumidor processa sua etapa de forma independente e registra um evento no banco de dados; falhas recorrentes são automaticamente encaminhadas para uma Dead Letter Queue (DLQ).

2.2 Tecnologias Utilizadas

Categoria	Tecnologia	Papel no sistema
Linguagem	Python 3.11+	Implementação de todos os serviços
API REST	FastAPI	Recepção e orquestração de pedidos
Mensageria	Amazon SQS (LocalStack)	Comunicação assíncrona entre serviços
Banco de dados	PostgreSQL 16	Persistência de pedidos e eventos
Nuvem emulada	LocalStack 3.4	Emulação local dos serviços AWS
Infra como código	Terraform	Provisionamento de filas, bucket e parâmetros
API externa	ViaCEP	Validação e enriquecimento de endereço
Containerização	Docker	Execução do LocalStack e PostgreSQL

2.3 Infraestrutura como Código (Terraform)

Toda a infraestrutura de mensageria e armazenamento foi descrita declarativamente em Terraform, eliminando a necessidade de criação manual de recursos. O trecho a seguir mostra a criação das cinco filas de serviço com política de redirecionamento automático para a Dead Letter Queue após três tentativas falhas:

```
resource "aws_sqs_queue" "filas" {
  for_each = toset(local.servicos)
  name     = "sqs-pedidos-${each.key}"

  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.dlq.arn
    maxReceiveCount     = 3
  })
}
```

```
}  
}  
}
```

O uso do laço `for_each` evita a repetição de código para as cinco filas (pagamento, estoque, fiscal, logística e notificação), e o bloco `redrive_policy` delega ao próprio SQS a responsabilidade de mover mensagens problemáticas para a DLQ, sem necessidade de lógica adicional nos consumidores.

2.4 Serviço de Pedidos (API REST)

O serviço de pedidos, implementado em FastAPI, expõe o endpoint `POST /pedidos`. Ao receber uma requisição, o serviço executa três operações em sequência: (1) consulta a API ViaCEP para validar o CEP informado e obter o endereço completo; (2) persiste o pedido no PostgreSQL com status `PENDENTE`; (3) publica o payload do pedido nas cinco filas SQS. A validação de CEP é assíncrona, utilizando a biblioteca `httpx`, o que evita bloquear o servidor enquanto aguarda a resposta da API externa:

```
async def consultar_cep(cep: str) -> dict | None:  
    url = VIACEP_URL.format(cep=cep_limpo)  
    async with httpx.AsyncClient(timeout=5.0) as client:  
        resposta = await client.get(url)  
        dados = resposta.json()  
        if dados.get("erro"):  
            return None  
        return dados
```

Caso o CEP seja inválido ou inexistente, a API retorna `HTTP 400` antes de qualquer escrita no banco ou publicação de mensagens, evitando o processamento de pedidos com endereço inconsistente.

2.5 Mensageria com Amazon SQS

Diferentemente do RabbitMQ, que possui exchanges com fan-out nativo, o SQS padrão exige a publicação explícita em cada fila de destino. A função `publicar_em_todas_filas` replica a mensagem nas cinco filas dos serviços consumidores:

```
def publicar_em_todas_filas(pedido: dict) -> None:  
    sqs = get_sqs_client()  
    corpo = json.dumps(pedido, ensure_ascii=False)  
    for url in TODAS_AS_FILAS:  
        sqs.send_message(QueueUrl=url, MessageBody=corpo)
```

O consumo das filas utiliza long polling (WaitTimeSeconds=10), reduzindo o número de chamadas vazias à API em comparação ao polling curto tradicional. Cada consumidor implementa uma função de callback que retorna um valor booleano: True remove a mensagem da fila (confirmação de processamento bem-sucedido) e False mantém a mensagem para nova tentativa, respeitando o limite de três tentativas definido na `redrive_policy` do Terraform.

2.6 Persistência em PostgreSQL

Foram criadas duas tabelas: `pedidos`, que armazena o estado atual de cada pedido, e `eventos_pedido`, que registra a linha do tempo de processamento de cada serviço. A separação permite reconstruir o histórico completo de um pedido — útil tanto para depuração quanto para a demonstração em sala — sem sobrescrever informações de etapas anteriores:

```
CREATE TABLE eventos_pedido (  
  id SERIAL PRIMARY KEY,  
  pedido_id VARCHAR(36) REFERENCES pedidos(id),  
  servico VARCHAR(50),  
  status VARCHAR(50),  
  mensagem TEXT,  
  criado_em TIMESTAMP DEFAULT NOW()  
);
```

2.7 Tratamento de Falhas e Dead Letter Queue

Para simular cenários realistas de instabilidade, o serviço de pagamento recusa aleatoriamente 10% das transações e rejeita compras acima de R\$ 5.000,00 quando a forma de pagamento é cartão de crédito. Quando um consumidor retorna False, a mensagem permanece na fila e é reentregue após o período de visibilidade configurado. Após três tentativas sem sucesso, o próprio SQS — sem intervenção do código da aplicação — redireciona a mensagem para a fila `sqs-dead-letter`, conforme a política definida no Terraform.

3. Resultados

4. Conclusões

A equipe concluiu que a migração de um broker tradicional (RabbitMQ) para um serviço de nuvem gerenciado (SQS) altera significativamente a forma de pensar o fan-out de mensagens, exigindo replicação explícita em múltiplas filas na ausência de um mecanismo equivalente ao exchange topic. A adoção de Terraform também evidenciou as vantagens de versionar a infraestrutura junto ao código da aplicação, permitindo recriar o ambiente de demonstração de forma determinística.

5. Melhorias Futuras

- Implementar autenticação e autorização (JWT) nos endpoints da API.
- Substituir o fan-out manual por SNS + SQS, aproximando a solução de um cenário de produção real.
- Migrar a infraestrutura do LocalStack para uma conta AWS real, validando custos e latência.
- Adicionar um painel de observabilidade para acompanhar filas e tempos de processamento em tempo real.

6. Referências

LocalStack Documentation.

Disponível em: <https://docs.localstack.cloud>. Acesso em: 13 de junho 2026.

Terraform Documentation.

Disponível em: <https://developer.hashicorp.com/terraform/docs>. Acesso em: 13 de junho 2026.

Amazon Simple Queue Service Developer Guide.

Disponível em: <https://docs.aws.amazon.com/sqs>. Acesso em: 13 de junho 2026.

FastAPI Documentation.

Disponível em: <https://fastapi.tiangolo.com>. Acesso em: 13 de junho 2026.

Webservice de Consulta de CEP.

Disponível em: <https://viacep.com.br>. Acesso em: 13 de junho 2026.

PostgreSQL Documentation.

Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 13 de junho 2026.

Teste a AWS infraestrutura usando LocalStack e Terraform Tests.

Disponível em: https://docs.aws.amazon.com/pt_br/prescriptive-guidance/latest/patterns/test-aws-infra-localstack-terraform.html. Acesso em 18 de junho 2026.

Manual de uso da API Busca CEP

Disponível em: <https://www.correios.com.br/atendimento/developers/manuais/manual-api-busca-cep>. Acesso em 18 de junho 2026.

Supabase Documentation

Disponível em: <https://supabase.com/docs>. Acesso em 18 de junho 2026.